Pump Condition Monitoring Project Documentation

Overview

This project is a synthetic predictive-maintenance demo for monitoring a pump asset. It generates telemetry, trains a Random Forest classifier on generated scenarios, stores readings in SQLite, exposes inference through FastAPI, and visualizes trends in a Streamlit dashboard. It is intentionally a monitoring-only educational project and does not control real equipment.

Project Purpose

The application demonstrates an end-to-end equipment monitoring pipeline using artificial pump operating conditions: normal, bearing wear, misalignment, cavitation, overload, and low flow. The repository is designed for portfolio presentation and local experimentation.

Tech Stack

Python 3.10+, FastAPI, Uvicorn, Pydantic, Streamlit, SQLite, scikit-learn, pandas, NumPy, joblib, pytest, requests.

Architecture

The project creates synthetic telemetry, trains a classifier using a synthetic dataset, persists sensor readings, exposes the model via REST API, and displays aggregated trends through a dashboard.

Core Components

Simulator generates feature vectors and scenario labels; detector trains and predicts with a Random Forest model; storage manages SQLite persistence; service ingests telemetry and attaches predictions; API exposes health, telemetry, and summary endpoints; dashboard visualizes trends and classifications; CLI seeds demo data and trains the model.

Key Files

README.md explains setup and usage. requirements.txt declares dependencies.
pump_monitoring/simulator.py generates telemetry features and scenarios. pump_monitoring/detector.py trains and predicts using the model. pump_monitoring/storage.py manages SQLite database writes and summaries. pump_monitoring/service.py orchestrates prediction and persistence. pump_monitoring/api.py exposes the FastAPI app. pump_monitoring/dashboard.py provides the Streamlit UI. pump_monitoring/cli.py seeds synthetic database records. pump_monitoring/train_model.py trains and saves the model and metrics. tests/test_core.py validates simulator, detector, storage, service, and API behavior.

Installation

Create a virtual environment, install dependencies with pip install -r requirements.txt, then train the model with python -m pump_monitoring.train_model.

Run Pipeline

Seed sample readings with python -m pump_monitoring.cli seed --count 240 --clear. Start the API with uvicorn pump_monitoring.api:app --reload --port 8001. Start the dashboard with streamlit run pump_monitoring/dashboard.py.

API Endpoints

GET /health returns API status and model state. POST /telemetry accepts a telemetry payload and stores the prediction. GET /telemetry returns recent readings. GET /summary returns aggregate stats.

Data Model

Telemetry fields include timestamp, asset_id, vibration_rms_mm_s, bearing_temp_c, motor_current_a, flow_m3_h, discharge_pressure_bar, rpm, plus predicted_condition, anomaly_score, severity, and explanation. All values are synthetic and intentionally constrained for demo use.

Testing

The repository includes a pytest suite covering simulator output, dataset generation, model training, persistence, service ingestion, and end-to-end API flow.

Important Limitations

The training and telemetry are synthetic and not based on real pump equipment. The classifier is not validated for real-world maintenance decisions, safety decisions, or equipment control. It is strictly a monitoring-only demonstration for learning and portfolio presentation.

Summary

This project demonstrates how a synthetic pump-monitoring app can combine data generation, model training, API inference, persistence, and dashboard visualization in a compact, well-structured Python application.